UNIVERSITI SAINS MALAYSIA

School of Electrical and Electronics Engineering
Universiti Sains Malaysia, Engineering Campus

SEMESTER 1, 2026/2027

EEM 441

Control & Instrumentation Laboratory

Experiment 7

DC MOTOR SPEED MEASUREMENT

Date : _____

Group No. : _____

Group Members : _____ Matrix # _____

: _____ Matrix # _____

: _____ Matrix # _____

: _____ Matrix # _____

Experiment 7

DC Motor Speed Measurement System

Objectives

1. To understand the methods of acquiring measurements automatically with the aid of a computer.
2. To become familiar with the data-acquisition (DAQ) environment.
3. To become familiar with signal conditioning in a computer-based data-acquisition system.
4. To design and build a signal-conditioning circuit whose parameters are justified by a measurement, rather than assumed.
5. To implement a computer-based measurement system for the rotational speed of a DC motor.

Equipment / Software Required

- DC motor trainer with a 12-slot wheel encoder
- Digital storage oscilloscope (DSO)
- Components for the student-designed filter and signal-conditioning circuit (op-amps, resistors, capacitors)
- ADAM-3968 wiring terminal board
- PCIE-1750U isolated digital I/O card, installed in the host PC
- PC with a C/C++ or Python development environment

Introduction

In most measurement processes it is often necessary to acquire data automatically, without a human operator. A computer-based instrument has several advantages over conventional measuring instruments: it can acquire measurements quickly and accurately, sustain real-time control, and operate for extended periods with consistent precision.

A computer-based measurement system typically contains the following elements (Figure 7.1): a transducer, a signal-conditioning stage, data-acquisition hardware, analysis hardware, and software running on the host PC.

Signal conditioning means manipulating an analogue signal so that it meets the requirements of the next processing stage — most commonly, so that it is suitable for an analogue-to-digital converter. Signal conditioning can include amplification, filtering, range matching, and isolation.

Filtering is the most common signal-conditioning function: frequencies that do not carry valid data are attenuated. In this experiment the encoder produces a pulse train whose useful frequency content is bounded, and any filter used ahead of the DAQ card should be designed — not assumed — around that bound.

Amplification increases the resolution of the input signal and improves its signal-to-noise ratio.

Isolation allows a signal to pass from its source to the measurement device without a direct physical (galvanic) connection, protecting sensitive equipment from the signal source and vice versa.

The Sallen–Key topology is a widely used active-filter structure, valued for its simplicity: it realises a two-pole (12 dB/octave) low-pass, high-pass, or band-pass response using a single op-amp configured as a near-unity-gain voltage-controlled voltage source (VCVS), together with two resistors and two capacitors that set the cutoff frequency and Q.

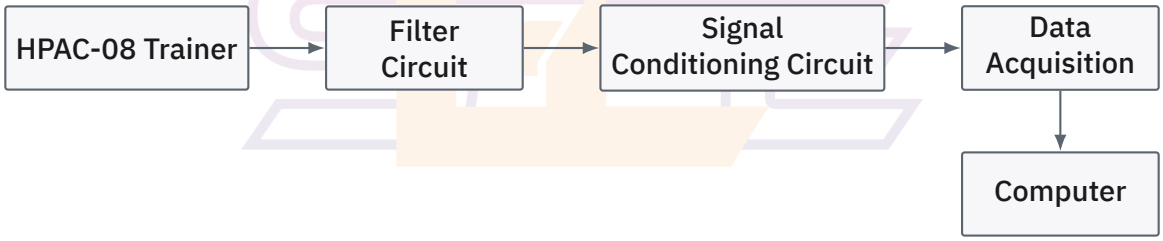


Figure 7.1: Computer-based measurement system: transducer, signal conditioning, DAQ hardware, and analysis software.

Procedure

Activity 1 — Characterising the encoder signal and designing the filter

Objective: measure the encoder's pulse characteristics and design a signal-conditioning filter around the measured cutoff frequency, rather than an assumed one.

1. Wire the encoder's output test point to the oscilloscope.
2. Run the motor across its usable speed range and observe the encoder pulse train on the oscilloscope.
3. At a representative operating speed, measure the pulse period and estimate the corresponding fundamental frequency from the oscilloscope trace.
4. From this measurement, determine a suitable cutoff frequency for noise-rejection filter.

5. Design the necessary filter based on the cutoff frequency determined prior.Justify your design choices.
6. Design the signal-conditioning circuit needed to present the filtered encoder pulses to the PCIE-1750U's digital input with valid logic levels. Justify your design choices.

Activity 2 — Interfacing to the DAQ hardware

Objective: connect the conditioned encoder signal to the PC via the ADAM-3968 and PCIE-1750U.

1. Connect the output of your signal-conditioning circuit to a digital input terminal on the ADAM-3968 terminal board.
2. Verify the voltage needed by PCIE1750U for it to detect HIGH/LOW logic using the NAVIGATOR application in the PC before writing any code, to confirm the expected pulses are being registered.



Figure 7.2: Signal path from the DC motor trainer to the host PC.

Activity 3 — Speed measurement software

Objective: implement software to compute and display motor speed (frequency and RPM) in real time, using two selectable measurement methods.

Two complementary methods can be used to convert the pulse train into a frequency and rotational speed, and your software should let the user select between them:

Method 1 — by counts (fixed pulse count, 1–99): the user specifies a target pulse count N ($1 \le N \le 99$); the software measures the elapsed time T_N for N pulses to occur, then computes

$$f = \frac{N}{T_N \times 12}\ [\text{rev/s}], \qquad \text{RPM} = 60f, \qquad \omega = 2\pi f\ [\text{rad/s}]$$

This method gives better resolution at high speed (many pulses arrive quickly, so T_N is measured over a longer, more precise interval) but responds more slowly at low speed, since a full count of N pulses can take a long time.

Method 2 — by period (fixed period window, 1–9 s): the user specifies a measurement window T seconds ($1 \le T \le 9$); the software counts the number of pulses N_T that occur within that window, then computes

$$f = \frac{N_T}{T \times 12}\ [\text{rev/s}], \qquad \text{RPM} = 60f, \qquad \omega = 2\pi f\ [\text{rad/s}]$$

In both methods, t_{start} and t_{stop} (marking the beginning and end of the measurement) are obtained from the DAQ card's counter/timer using

$$\text{Elapsed time} = \frac{t_{\text{stop}} - t_{\text{start}}}{\text{CLK_TCK}}$$

where CLK_TCK is the counter's clock tick rate.

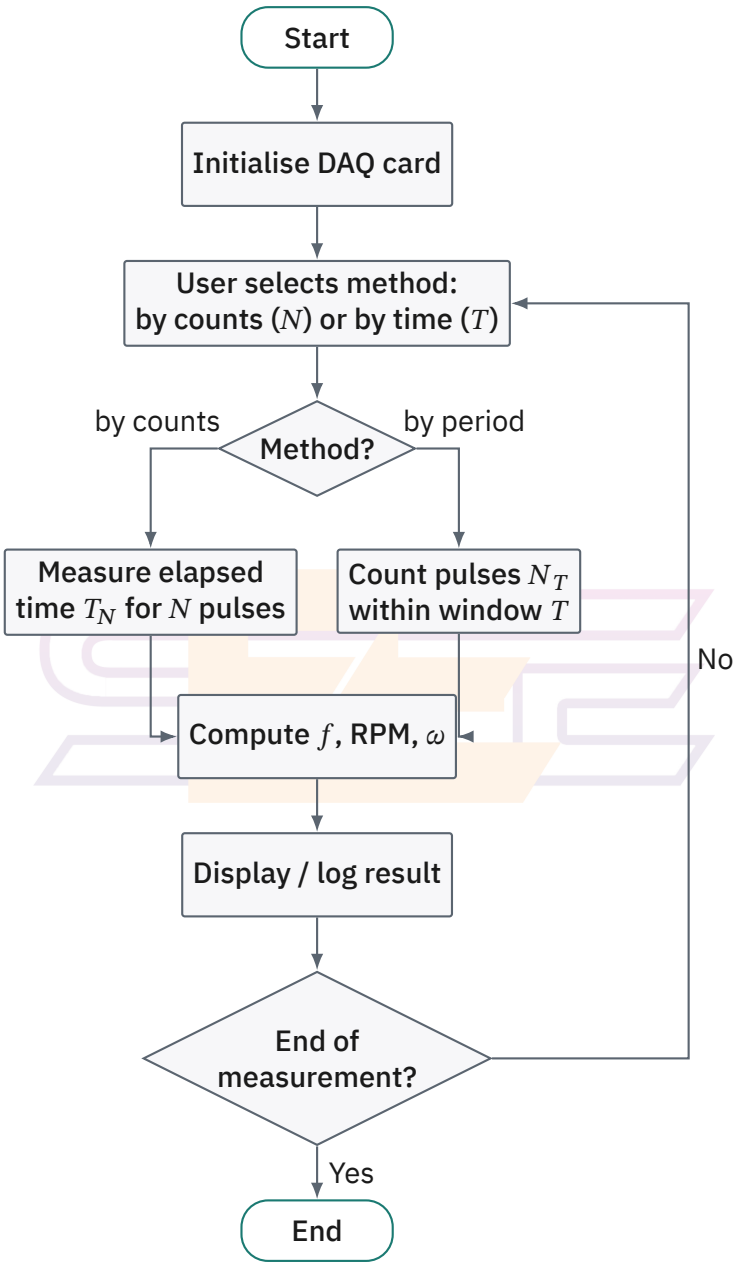


Figure 7.3: Measurement loop for the speed-acquisition software

1. In C, C++, or Python, write a program that:
   - initialises the PCIE-1750U (via the vendor driver / SDK, or the DAQ card's counter API),
   - lets the user select the measurement method (by counts, $1 \le N \le 99$; or by time, $1 \le T \le 9$ s) and enter the corresponding parameter,
   - for the *by counts* method: times how long it takes for N encoder pulses to occur,

- for the *by time* method: counts how many encoder pulses occur within the window T,
- computes the frequency, RPM, and angular speed ω using the relations above, and
- displays the running value (frequency and RPM, with the active method indicated) in a simple GUI, updating in real time.

2. Interface your filter/conditioning circuit, the ADAM-3968, and the PCIE-1750U together and perform measurements across a range of motor input voltages, using both methods at each operating point. Compare your computed speed against a direct oscilloscope measurement of the encoder period at the same operating point.
3. Tabulate your results (Tables 7.1–7.4).
4. Plot computed speed (from both software methods) against the oscilloscope-measured speed.
5. Discuss and comment on your results, including any discrepancy between the two software methods and the oscilloscope reference, and how each method's resolution changes with motor speed (see the trade-off noted above).

Table 7.1: By-counts method: frequency comparison table.

N	Measured Frequency (Hz)					Oscilloscope (Hz)
	#1	#2	#3	#4	#5	

Verified by

Date:

Table 7.2: By-counts method — RPM comparison table.

N	ω (Measured RPM)					Oscilloscope (calculated RPM)
	#1	#2	#3	#4	#5	

Verified by

Date:

Table 7.3: By-period method: frequency comparison table.

T (s)	Measured Frequency (Hz)					Oscilloscope (Hz)
	#1	#2	#3	#4	#5	

Verified by

Date:

Table 7.4: By-period method: RPM comparison table.

T (s)	ω (Measured RPM)					Oscilloscope (calculated RPM)
	#1	#2	#3	#4	#5	

Verified by

Date:

Report

1. Present your oscilloscope measurements of the encoder pulse and your resulting filter design (topology, calculations, and component values), together with the measured response of the built filter.
2. Present your design of the signal-conditioning stage between the filter and the PCIE-1750U digital input, with justification.
3. Compare your software-computed speed (both by-counts and by-time methods) with the oscilloscope-measured speed across your tested operating points, and discuss the sources of any discrepancy.
4. Compare the by-counts and by-time methods directly: at what motor speeds does each method give better resolution or a more stable reading, and why? Which would you recommend for a low-speed application, and which for a high-speed one?
5. What is the theoretical maximum measurable speed of this system, given the encoder resolution (12 slots/revolution) and your software's timing resolution?

Deliverables

1. **Code:** a programming script that accurately measures and displays the temperature based on sensor inputs, and controls the heater accordingly.
2. **System setup diagram:** a schematic diagram of your circuitry and wiring diagram of the system.
3. **Documentation:** a report that explains the setup, the temperature control process, sample readings, discussion, and any challenges encountered — including your justification for omitting the DAC0800 (see Experimental Equipment).
4. **Demo:** a short presentation or video demonstrating the working system and the collected data.
5. **Video:** a video of your work, showing the encoder signal, your filter and signal-conditioning stage, and your software displaying the measured motor speed as the speed is varied.

Appendix: PCI-1750/PCI-1750SO Reference

The PCI-1750/PCI-1750SO offers 16 isolated digital input channels, 16 isolated digital output channels, one isolated counter, and one timer with a PCI bus interface. With isolation protection of 2500 VDC, the PCI-1750/PCI-1750SO is ideal for industrial applications where high-voltage protection is required.

The card's 16 bits are divided into two 8-bit I/O ports, which makes the PCI-1750/PCI-1750SO very easy to program. The card also offers dual interrupt handling capability, giving the user more flexibility in using the counter, timer, digital inputs, or a combination of these to generate interrupts to the PC; interrupts are configured entirely through software.

The PCI-1750/PCI-1750SO uses a PCI controller to interface the card to the PCI bus. The controller fully implements the PCI bus specification Rev. 2.1, and all bus-relative configurations — such as base addresses and interrupt assignments — are automatically handled by software. Figures 7.4–7.6 show the relevant reference information for the PCI-1750 card; refer to the PCI-1750 user manual for further detail on the card and the functions available in its DAQ software.

PCI-1750/PCI-1750SO

32-ch Isolated Digital I/O and 3-ch Counter PCI Card



Main Features

- 16-ch isolated digital input and 16-ch isolated digital output channels
- High voltage isolation on all isolated channels (2,500 V_{DC})
- High sink current on isolated output channels (200 mA/Channel)
- D-type 37-pin female connector
- Supports dry contact or 5 ~ 50 V_{DC} isolated input
- Dual interrupt handling capability
- Timer/Counter interrupt capability

Figure 7.4: PCI-1750/PCI-1750SO: card overview and main features.

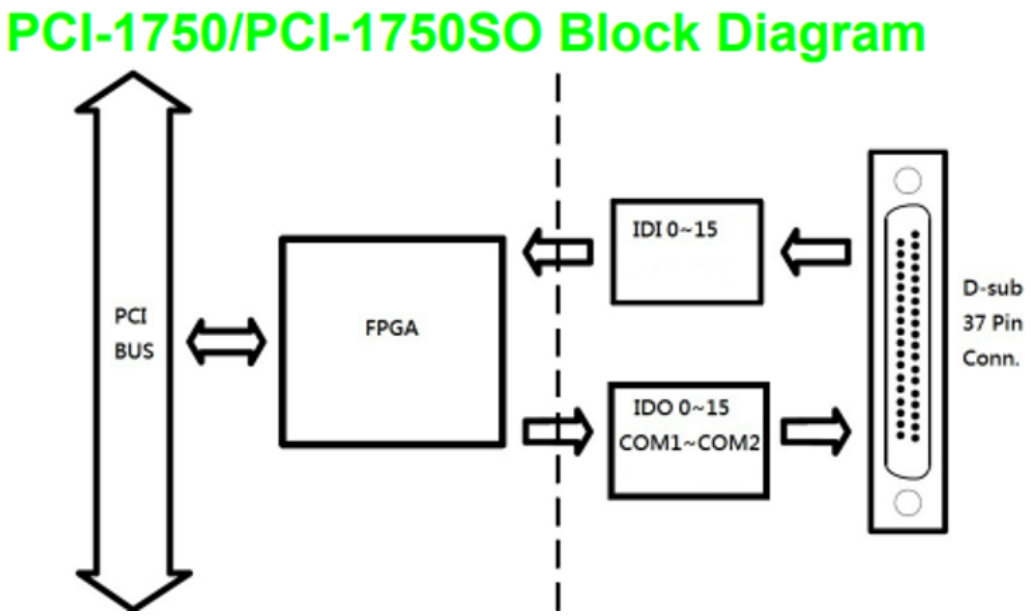


Figure 7.5: PCI-1750/PCI-1750SO block diagram (PCI-1750 user manual, p. 9).

INT0 / IDI 0	1	20	IDI 1
IDI 2	2	21	IDI 3
INT0_G / IDI 4	3	22	IDI 5
IDI 6	4	23	IDI 7
INT1 / IDI 8	5	24	IDI 9
IDI 10	6	25	IDI 11
INT1_G / IDI 12	7	26	IDI 13
IDI 14	8	27	IDI 15/Counter2
IGND	9	28	IGND
COM1	10	29	IGND
IDO 0	11	30	IDO 1
IDO 2	12	31	IDO 3
IDO 4	13	32	IDO 5
IDO 6	14	33	IDO 7
IDO 8	15	34	IDO 9
IDO 10	16	35	IDO 11
IDO 12	17	36	IDO 13
IDO 14	18	37	IDO 15
COM2	19		

Figure 7.6: PCI-1750/PCI-1750SO: I/O pin assignment.